

Appendix A

Selected Conversation Logs — GenAI Usage

The following excerpts demonstrate how GitHub Copilot (Claude model) was used throughout the EveryMovie project. Conversations are grouped by usage category as described in Section 7 of the Technical Report.

1. Architecture & Planning

Early conversation about how to structure the Django project and decide what data to store locally versus fetching from TMDB on demand.

User: I want to build a movie discovery app using TMDB API. Users should be able to browse movies by genre and country, search for movies, see trending films, and create personal movie lists. What's the best way to structure this in Django?

Copilot: I'd recommend a single Django app (EveryMovie) inside a config project. For data storage, only store what users create — movie lists and list items. All movie metadata (titles, posters, ratings, cast) should be fetched from TMDB on demand. This keeps your database lightweight and avoids syncing issues. Your models would be `MovieList` (linked to `User`) and `MovieListItem` (linked to `MovieList`, storing just the `tmdb_id`, `title`, and `poster_path`).

User: Should I build separate JSON API endpoints or just render everything server-side?

Copilot: A hybrid approach works well here. Serve the main pages via Django templates, but expose JSON endpoints for data that the frontend fetches dynamically: `/genres/`, `/discover/`, `/search/`, `/api/trending/`, and `/api/lists/`. The frontend JavaScript can call these with `fetch()` to update the page without full reloads. This also satisfies the coursework requirement for a data-driven web API with JSON responses.

2. Code Implementation

Excerpt from implementing the genre discovery feature with dynamic pair detection.

User: I want the genre chips to show not just single genres like 'Action' but also genre pairs like 'Action/Comedy' if that combination actually exists in real movies. How should I implement this?

Copilot: The `/genres/` endpoint can sample popular and top-rated movies from TMDB and extract all two-genre combinations that appear. For each movie with multiple genres, generate all pairs. Collect these into a set, then return them alongside the single genres. This way the pairs are always realistic — they reflect what TMDB actually has rather than being hard-coded.

User: Can you implement this in the genres view?

Copilot: Here's the approach: fetch `/genre/movie/list` from TMDB for the genre ID map, then sample movies from `/discover/movie` sorted by popularity and `vote_count`. For each movie, extract its `genre_ids` and generate pairs using `itertools.combinations`. Return both singles and pairs in the JSON response with a structure like `{"genres": [{"id": "28", "name": "Action"}, {"id": "28,35",`

```
"name": "Action/Comedy"}, ...}}.
```

Implementing the movie lists CRUD functionality.

User: I need authenticated users to be able to create movie lists, add movies to them, rename and delete lists, and remove individual items. Can you implement the full CRUD?

Copilot: I'll create views for each operation: `create_list` (POST with name), `rename_list` (POST with new name), `delete_list` (POST to confirm), `add_to_list` (POST with `tmdb_id`, title, poster_path), and `remove_from_list` (POST to delete item). All will be protected with `@login_required`. The `api_lists` endpoint will return all lists for the authenticated user as JSON, or 401 for unauthenticated requests.

3. Problem Solving

Debugging the genre pair gradient rendering issue.

User: The genre pair chips are supposed to show a gradient of two colours (one for each genre) but when I apply `border-image` for the gradient it completely breaks the `border-radius` and the chips become square instead of pills.

Copilot: This is a known CSS limitation — `border-image` overrides `border-radius` in all major browsers. The fix is to use a background gradient instead. Apply a linear-gradient as the background property with the two genre colours, and keep `border-radius` as normal. For example: `background: linear-gradient(135deg, #ff6b6b 50%, #4ecdc4 50%);` with `border-radius: 999px`.

Fixing the dynamic quality filtering issue for rating-based sorts.

User: When I sort movies by rating for a specific country and genre, obscure movies with like 1 vote and a 10/10 rating show up at the top. How do I fix this?

Copilot: You need a minimum vote count threshold, but it should be dynamic. First, probe TMDb to get the `total_results` count for that genre/country combination. Then set the vote count threshold proportionally — for example, if there are 5000+ results, require `vote_count >= 100`; if there are only 50 results (niche combination), require `>= 5`. This way popular genre/country combos show genuinely rated films while niche ones still return results.

Mobile layout issues with the genre scatter effect.

User: The genre chips use absolute positioning with a scatter algorithm on desktop which looks cool but on mobile the chips move around when scrolling and overlap each other. How can I fix the mobile experience?

Copilot: For mobile, replace the scatter layout with a simple alphabetical flex-wrap layout. Use a CSS media query for screens under 768px to set the container to `display: flex` with `flex-wrap: wrap` and a `max-height` with `overflow-y: auto`. In JavaScript, use `window.matchMedia` to detect mobile

and skip the scatter algorithm, just sorting alphabetically and appending chips in flow order.

User: Actually, let's just remove the scatter entirely for all screen sizes. The alphabetical flex layout is cleaner.

Copilot: Done — I've replaced the scatter algorithm with alphabetical sorting and simple DOM appends for all screen sizes. Removed the collision detection code, random positioning, and mobile checks. The chips now use flex-wrap layout everywhere with a max-height and scroll on mobile.

4. Deployment

Setting up and debugging the Render deployment.

User: I want to deploy on Render. What do I need to set up?

Copilot: You'll need: 1) A requirements.txt (run `pip freeze > requirements.txt`). 2) Gunicorn for the WSGI server (`pip install gunicorn`). 3) WhiteNoise for static files. 4) Environment variables on Render: `TMDB_API_KEY`, `DEBUG=False`, `ALLOWED_HOSTS=your-app.onrender.com`. Build command: `pip install -r requirements.txt && python manage.py collectstatic --noinput`. Start command: `gunicorn config.wsgi:application`.

Diagnosing 500 errors on login/signup after deployment.

User: The site is deployed but when I try to log in or sign up I get a 500 error. It works fine locally.

Copilot: This is likely a CSRF issue. Django 4+ requires `CSRF_TRUSTED_ORIGINS` to be set when behind a reverse proxy (which Render uses). Add this to `settings.py`: `CSRF_TRUSTED_ORIGINS = [f"https://{host}" for host in os.environ.get("ALLOWED_HOSTS", "").split()]`. Also, make sure your Render build command includes `'python manage.py migrate'` — without it, the SQLite database has no tables and auth will fail.

User: That fixed it! Login and signup both work now.

5. Documentation

Creating the API documentation and technical report.

User: I need API documentation as a PDF. Can you create comprehensive docs for all my endpoints?

Copilot: I'll document all 12 endpoints: 5 JSON APIs (`/genres/`, `/discover/`, `/search/`, `/api/trending/`, `/api/lists/`), 5 list CRUD endpoints, and 2 auth endpoints. Each will include the HTTP method, URL, parameters, authentication requirements, example request, example response, and error codes. I'll also include data models for `MovieList` and `MovieListItem`, and document the external TMDB API usage.

Shortening the technical report to meet the 5-page limit.

User: I updated the technical report but it's too long — 8 pages. The brief says max 5 pages. How do I cut it down while meeting all requirements?

Copilot: The biggest wins: 1) Replace the Introduction's bullet-point feature list with 2–3 sentences — features are already in Architecture. 2) Replace the 8-row test table with a paragraph summarising the three test categories. 3) In Architecture, replace the database schema table with inline prose. 4) In Tech Stack, remove the FastAPI comparison paragraph. 5) Keep GenAI section almost intact — it's mandatory and your reflection is strong. These cuts should bring it from 8 to 5 pages.

6. Testing

Writing automated tests for the API endpoints.

User: I need automated tests for the coursework. What should I test?

Copilot: Focus on three areas: 1) Input validation — test that endpoints return 400 when required parameters are missing (e.g. /search/ without q, /discover/ without genre and country). 2) Authentication enforcement — verify /api/lists/ returns 401 for unauthenticated users. 3) End-to-end flows — signup, login, create a list, then verify it appears in the API response. For tests that call TMDB, accept 502/503 as valid so tests don't fail if TMDB is temporarily down.

Verifying password security.

User: Is my password hashing secure? Do I need to add anything?

Copilot: Django's create_user() already uses PBKDF2-SHA256 with a random salt per user and 870,000 iterations (Django 6 default). This meets OWASP recommendations. No changes needed — your auth is already secure out of the box.

End of Appendix A